Reto HTML + Tailwind — Nivel Extremo

"cuando las herramientas son pocas,
el dominio del lenguaje y la creatividad hacen el resto"

Feed de red social · Duración máxima: 60 minutos · Trabajo individual

Introducción.
El último caso práctico de *DataTech Solutions* es el feed de una red social, como el de Instagram o X: muchísimo contenido, de tipos distintos y consumido sobre todo desde el celular. Antes de conectar cualquier backend, el equipo de producto quiere un prototipo navegable de alta fidelidad. La condición del líder técnico es radical: **cero JavaScript**. El menú lateral, las pestañas, los likes, los comentarios desplegables y la ventana modal deben funcionar solo con HTML nativo y variantes de Tailwind CSS. Este reto mide qué tan profundo dominas el lenguaje, no cuántas librerías conoces.

Objetivo general.
Construir un prototipo interactivo y responsive del feed de una red social usando únicamente HTML y Tailwind CSS, resolviendo la interactividad con estados nativos del navegador (`:checked`, `:target`, `[open]`, `:focus-visible`) en lugar de JavaScript.

Objetivos específicos.

- Diseñar un layout de tres zonas que se transforma por completo entre móvil y escritorio.
- Implementar interactividad sin JavaScript con el patrón checkbox/radio + `peer`, el elemento `details` y la pseudoclase `:target`.
- Usar peers con nombre (`peer/nombre`) para controlar varios estados independientes en un mismo contenedor.
- Mantener la accesibilidad: todo se opera con teclado, los botones de solo ícono tienen `aria-label` y se respeta la preferencia de movimiento reducido.

Reglas del reto.

- Un único archivo `index.html`. Tailwind CSS v4 se carga solo con el script oficial del CDN (ver punto de partida); es el único `<script>` permitido.
- Prohibido: la etiqueta `<style>`, el atributo `style=""`, archivos `.css`, JavaScript propio (incluidos atributos como `onclick`) y librerías de componentes (DaisyUI, Flowbite, Preline, etc.).

- Imágenes: usa `https://picsum.photos/seed/<nombre>/600/600` o íconos SVG escritos en línea.
- Tiempo máximo: **60 minutos**. Al terminar el tiempo se entrega lo que funcione; es mejor una parte impecable que todo a medias.
- En este nivel sí se permiten variantes y propiedades arbitrarias (ej. `[&::-webkit-scrollbar]:hidden`), pero nunca la etiqueta `<style>`.

Punto de partida.
Copia esta estructura en tu `index.html` y construye todo dentro del `<body>`:

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Feed | DataTech Social</title>
  <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
</head>
<body class="bg-slate-50 text-slate-800 antialiased">
  <!-- Tu solución va aquí -->
</body>
</html>
```

Qué se espera de la solución.
Un prototipo que parezca una app real tanto en 360 px como en 1440 px, donde cada interacción descrita funcione con el mouse y también con el teclado. Además, el coder deberá responder en su README:

- ¿Qué partes de tu prototipo necesitarían JavaScript o un backend en producción y por qué (por ejemplo, que el like se conserve al recargar)?
- Retomando el caso práctico del día: ¿qué datos de este feed guardarías en una base clave-valor (caché) y cuáles en una documental?

El coder debe ejecutar y documentar los siguientes pasos:

1. **Layout de tres zonas (12 min):** en `xl` va una navegación a la izquierda, el feed al centro (`max-w-xl`) y una columna de tendencias a la derecha; en `lg` desaparece la columna derecha. En móvil solo se ve el feed, con un header superior y una barra de navegación inferior fija (`fixed bottom-0`) que no tape el último post.
2. **Menú lateral móvil (10 min):** un checkbox oculto (`sr-only peer`) controla un drawer: el `aside` entra desde la izquierda (`-translate-x-full`

peer-checked:translate-x-0 con transición) y aparece un overlay oscuro que también es un label para cerrarlo. Usa motion-reduce:transition-none.

3. **Historias y pestañas (12 min):** una fila de historias con scroll horizontal y snap (snap-x snap-mandatory), sin barra de scroll visible. Debajo, las pestañas «Para ti» y «Siguiendo» con dos radios hermanos de los paneles: usa peers con nombre (peer/para-ti, peer/siguiendo) para mostrar el panel correcto y resaltar la pestaña activa.
4. **Publicaciones y likes (14 min):** al menos 4 posts de tipos distintos (solo texto, con imagen, con galería 2x2 y con preview de enlace). El like es un checkbox: al marcarlo el corazón se rellena de rojo y el contador sube en uno (dos span que se alternan con peer-checked:hidden y hidden peer-checked:inline). Los comentarios se despliegan con details/summary y un ícono que rota con group-open:rotate-180.
5. **Modal de nueva publicación (12 min):** un botón flotante «Crear» (enlace a #nuevo-post) abre una modal con :target: la sección empieza hidden y pasa a target:flex, centrada sobre un fondo con backdrop-blur. Incluye un textarea con maxlength="280", botón Publicar y un enlace para cerrar (href="#").

Pistas técnicas (no son la solución).

- El orden del HTML importa: peer solo funciona hacia hermanos posteriores. Ubica los inputs antes de lo que controlan, aunque visualmente deban verse en otro lugar.
- Un label con for puede estar en cualquier parte del documento: úsalo como «botón» que activa un input lejano.
- Para ocultar la barra de scroll: [scrollbar-width:none] y [&::-webkit-scrollbar]:hidden.
- Para que la barra inferior fija no tape contenido, agrega padding inferior al main (pb-20 lg:pb-0).
- Para que se vea el foco en un label que envuelve un input sr-only: has-[:focus-visible]:ring-2.
- Al abrir la modal con :target la URL cambia a #nuevo-post; es el comportamiento esperado.

Reto extra (opcional, no suma tiempo).

- Un post de encuesta cuyas barras de resultado solo se llenan después de votar (group-has-[:checked]:w-[62%]).
- Modo oscuro con la variante dark: en todo el prototipo.

Criterios de evaluación.

Criterio	Peso
Layout responsive de tres zonas y barra inferior	20 %
Drawer móvil y modal con :target	20 %
Pestañas con peers con nombre	20 %
Posts heterogéneos, likes y comentarios	20 %
Accesibilidad (teclado, aria-label, motion-reduce)	10 %
README y respuestas de reflexión	10 %

Entregables.

- Enlace del repositorio de la actividad en GitHub.
- Archivo `index.html` con la solución.
- Archivo README.md estructurado que contenga:
  - Título y nombre del coder.
  - Un GIF o video corto (máx. 30 s) mostrando cada interacción en móvil y escritorio.
  - Explicación breve de la técnica usada para cada interacción (qué input o pseudoclase y qué variante).
  - Respuestas a las preguntas de reflexión.

Nombre Reto HTML + Tailwind — Nivel Extremo	**Creado por** Jorge Areiza
Fecha de Creacion 21 de octubre del 2026, Medellin	**Ultima actualizacion**
Version 1.0.0	**Comentarios** Duración máxima: 60 minutos. Solo HTML + Tailwind CSS v4 (CDN). Cero JavaScript. Trabajo individual.